

SPRINT 11 · AGENT FOUNDATIONS · SESIÓN 1

Del RAG al agente

Estado, un turno y quién decide cuándo terminamos. Resolvemos la Live Review en directo mientras repasamos la teoría.

HOY, EN 120 MINUTOS

Teoría y código en el mismo hilo

0-10	Apertura	De responder() a procesar_turno()
10-25	Pincelada 1	Qué es (y qué no es) un agente · quiz
25-55	Live Review · Fase 0	Entorno, estructura del proyecto y arquitectura
55-65	Descanso	
65-95	Pincelada 2 y 3	AgentState, tres memorias y JSON estructurado
95-120	Live Review · Fase 1	Implementamos procesar_turno y lo probamos

DE DÓNDE VENIMOS

Mismo LLM, otro contrato

Sprint 10 · RAG

`responder(pregunta) → dict`

Una pregunta, una pasada fija: recuperar → prompt → generar. No hay “dónde voy en la tarea”.

Sprint 11 · Agente

`procesar_turno(estado, mensaje) → estado`

Cada mensaje actualiza un estado que dirige la tarea hasta que Python decide que ha terminado.

Pregunta del sprint: **si ya sé responder con RAG en una llamada, ¿cómo planifico en varios pasos con estado y control?**

Cinco patrones que no son lo mismo

LLM	Genera texto en una llamada	Solo el prompt	No
Chatbot	Conversa turno a turno	Historial de mensajes	No planifica
Asistente	Rol o instrucciones fijas	A veces	1 respuesta por turno
Pipeline RAG	Recupera y genera una respuesta	No	Flujo fijo
Agente	Razona, actualiza estado y decide si parar	Sí	Sí: loop + parada

Un chat con historial **no** es automáticamente un agente.

PINCELADA 1 · QUIZ RÁPIDO

¿RAG, chatbot o agente?

A

“Según la documentación, ¿qué significa este campo?”

B

“Arma un plan de tarde: cine o música, 3 horas, zona centro.”

C

“Hola, ¿qué tal?”

D

“¿Cuál es el valor de NO₂ ahora mismo en Escuelas Aguirre?”

Respuestas: A → RAG · B → agente · C → chatbot · D → ni RAG ni agente sin datos: abstenerse.

Agente de calidad del aire en Madrid

El dominio

Orientar una consulta: **zona**, **contaminante** y **tipo de consulta**.
Cierra con un plan de consulta genérico. Sin RAG, sin tools, sin mediciones.

Lo que viene dado

state.py · llm.py · main.py · app.py ·
config.py · verificar.py

Lo que implementamos

procesar_turno() hoy y run_demo()
en la sesión 2, ambos en
src/agent.py.

Mismo patrón que el proyecto de la tarde cultural. Cambian los campos del estado; el loop no.

Arrancamos con el check en rojo

```
cd 06_AI_Agents_Engineering/Sprint_11/  
cd Practica_live_review/01_agente_calidad_aire  
python -m venv .venv  
source .venv/bin/activate  
# Windows: .venv\Scripts\Activate.ps1  
pip install -r requirements.txt  
cp .env.example .env # pega tu GEMINI_API_KEY  
python main.py --check
```

Salida esperada:

```
[FAIL] procesar_turno  
[FAIL] run_demo
```

Rojo es bueno: el verificador detecta que aún hay trabajo.

Cinco piezas, cinco archivos

Entrada	El mensaje del usuario	main.py (CLI) · app.py (Streamlit)
LLM	Propone cambios dado el estado	src/llm.py · preguntar_al_modelo
Estado	“Dónde vamos” en la tarea	src/state.py · crear_estado
Decisión	¿Seguimos o paramos?	calcular_done · max_turns
Salida	Respuesta, plan, error y traza	src/agent.py · procesar_turno

Un agente, en lo esencial: **loop + estado + parada + LLM**. El resto son capas.

El esquema mental, en seis líneas

```
estado = crear_estado(pedido)

for mensaje in mensajes[:max_turns]:
    if estado["done"] or estado["error"]:
        break
    cambios = preguntar_al_modelo(estados, mensaje)
    estado = actualizar_estado(estados, cambios)
    estado = calcular_done(estados)
```

Dentro del for = un turno → será procesar_turno.

El for con su tope = la conversación → será run_demo.

One-shot (RAG) no tiene este bucle.

La fuente de verdad de la tarea

```
{
  "objetivo": "Orientar una consulta (Madrid)",
  "pedido_original": "",
  "preferencias": {
    "zona": None,
    "contaminante": None,
    "tipo_consulta": None
  },
  "plan": [],
  "respuesta": "",
  "done": False,
  "error": None,
  "traza": []
}
```

Campos con nombre estable: prompt y código hablan el mismo idioma.

El LLM no es la memoria: lo que no escribes en el estado, se pierde.

done, error y max_turns los controla el código.

No llames “memoria” a cualquier dict

A · Historial de chat

`st.session_state.messages`

Pinta la conversación. Solo presentación.

B · Estado de ejecución

`agent_state`

Dirige la tarea: preferencias, plan, done, traza. Vive lo que dura la conversación.

C · Long-term

`preferencias.json`

Persiste entre sesiones. Opcional en este sprint.

Frase del sprint: **messages pinta el chat; AgentState dirige la tarea.**

El LLM propone. Python decide.

El modelo devuelve preferencias, plan y respuesta. Nunca devuelve done: lo calcula `calcular_done()` en código.

El modelo solo puede contestar en JSON

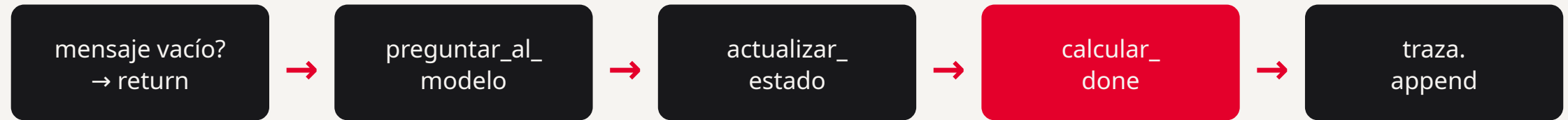
```
{
  "preferencias": {
    "zona": "centro",
    "contaminante": "NO2",
    "tipo_consulta": null
  },
  "plan": [],
  "respuesta": "¿Qué quieres saber del NO2?"
}
```

Reglas clave del prompt en `llm.py`:

- No inventes preferencias que el usuario no haya dicho.
- Sin zona, contaminante y tipo → plan vacío y pregunta.
- Plan solo tras confirmación del usuario.
- No inventes mediciones ni umbrales.
- No incluyas “done”.

FASE 1 · QUÉ HACE UN TURNO

procesar_turno(estado, mensaje)



Camino feliz

Una entrada en la traza con **status: ok** y el valor de done.

Camino de error

try/except: **error** en el estado, respuesta amigable y traza con **status: error**. Nunca se relanza.

Mensaje vacío

Respuesta amigable **sin** llamar al LLM y **sin** traza nueva. Ya viene hecho.

El núcleo que escribimos juntos

```
traza_actual = estado.get("traza", [])
n = len(traza_actual) + 1
try:
    cambios = preguntar_al_modelo(estado, mensaje)
    estado = actualizar_estado(estado, cambios)
    estado = calcular_done(estado) # Python decide done
    estado.setdefault("traza", []).append(
        {"turno": n, "mensaje": mensaje, "status": "ok", "done": estado.get("done")})
except Exception as e:
    estado = dict(estado)
    estado["error"] = str(e)
    if not estado.get("respuesta"):
        estado["respuesta"] = "No he podido continuar por un error técnico. ..."
    estado.setdefault("traza", []).append(
        {"turno": n, "mensaje": mensaje, "status": "error", "detail": str(e)})
return estado
```

FASE 1 · LO PROBAMOS

python main.py --interactivo

Guion de mensajes, turno a turno:

1. Quiero orientarme sobre calidad del aire en Madrid.
2. Me interesa el NO2 o la magnitud 83.
3. Zona centro, cerca de Escuelas Aguirre.
4. Quiero entender qué significa un valor alto.
5. Sí, adelante con el plan de consulta.

Qué mirar en cada turno:

- Cómo se rellenan las preferencias.
- Cuándo aparece el plan.
- En qué turno done pasa a true.

Si en el 4 ya hay done, el 5 no se envía: éxito temprano.

CIERRE DE LA SESIÓN 1

Tres ideas y un reto para la próxima sesión

1

Un agente es loop + estado + parada + LLM.

2

messages pinta el chat; AgentState dirige la tarea.

3

El LLM propone; Python decide done.

Reto: intentad implementar `run_demo()` antes de la próxima clase. Lo resolveremos juntos al principio.

SPRINT 11 · AGENT FOUNDATIONS · SESIÓN 2

Control: parada, errores y UI

Implementamos `run_demo`, rompemos el agente a propósito, lo llevamos a Streamlit y lo miramos con los ojos de LangGraph.

HOY, EN 120 MINUTOS

Del turno a la conversación controlada

0-10	Repaso relámpago	Cinco preguntas en frío
10-45	Parada + Fase 2	Condiciones de parada · run_demo en directo
45-60	Errores y traza	Rompemos la API key a propósito
60-70	Descanso	
70-95	Fase 3 · Streamlit	app.py como cliente · dos memorias
95-110	Panorama LangGraph	Nuestro código dibujado como grafo
110-120	Cierre	Criterio de cierre y puente al Sprint 12

REPASO RELÁMPAGO

Contestad en el chat, sin mirar apuntes

- 1 ¿Qué recibe y qué devuelve `procesar_turno`?
- 2 ¿Quién decide `done`: el modelo o Python?
- 3 ¿Qué tres preferencias necesita nuestro agente?
- 4 ¿Qué pasa si el usuario envía un mensaje vacío?
- 5 ¿Por qué el modelo no puede darnos el valor de NO_2 de ahora?

Un agente sin límites es un bucle caro

Objetivo cumplido	Solo código	calcular_done: tres preferencias + plan $\geq$ 2 ítems
max_turns	Solo código	Tope de mensajes de usuario → LLM (8 por defecto)
Error irrecuperable	Código	API key ausente o inválida, JSON roto
Sin progreso	Código (opcional)	Estado idéntico dos turnos seguidos

Parar es una feature, no un fallo. Nunca confíes en que el modelo “cierre” con palabras.

Éxito temprano frente a tope

Éxito temprano

```
python main.py
```

El guion tiene 5 mensajes, pero en el 4 ya hay `done=true`.
El 5 no se envía. **corte: ninguno (`done=true`)**

Corte por tope

```
python main.py --max-turns 2
```

Solo se procesan 2 mensajes. `done` sigue en `false` y se añade un aviso. **corte: `max_turns` (2)**

El guion propone pasos; el estado decide si se consumen.

run_demo: la conversación con tope

```
msgs = list(mensajes) if mensajes is not None else list(config.DEMO_MENSAJES)
limit = max_turns if max_turns is not None else config.MAX_TURNS_DEFAULT
estado = crear_estado(msgs[0] if msgs else "")

for mensaje in msgs[:limit]:
    if estado.get("done") or estado.get("error"):
        break
    estado = procesar_turno(estado, mensaje)

terminado = bool(estado.get("done") or estado.get("error"))
corto_por_limite = len(estado.get("traza", [])) >= limit or len(msgs) > limit
if not terminado and corto_por_limite:
    aviso = "He parado por max_turns; esto es lo que llevo del plan."
    prev = (estado.get("respuesta") or "").strip()
    estado["respuesta"] = f"{prev}\n\n{aviso}" if prev else aviso
return estado
```

FASE 2 · TRES COMANDOS

Lo que tiene que salir en la terminal

<code>python main.py</code>	<code>done: true · plan con ≥ 2 ítems · 4–5 turnos en la traza</code>
<code>python main.py --max-turns 2</code>	<code>done: false · corte: max_turns (2) · aviso en la respuesta</code>
<code>python main.py --check</code>	<code>[OK] procesar_turno · [OK] run_demo · [OK] mensaje vacío</code>

Leed siempre el bloque `--- resumen ---` antes que el JSON completo.

Si no puedes enseñar qué turnos dio, no es demostrable

API key o cuota	Guardar error; no reintentar en bucle
JSON inválido	leer_json limpia ``; si falla, abortar el turno
Campos que faltan	actualizar_estado ignora los None
Timeout o red	Un reintento con backoff (opcional)

Una entrada de traza por turno:

```
{"turno": 1,
  "mensaje": "Quiero orientarme...",
  "status": "error",
  "detail": "API key not valid..."}
```

Abortar frente a reintentar

Error de parseo

Reintentar **una vez** con “SOLO JSON”. Si sigue fallando: error en el estado y traza con status error.

Error de autenticación

No reintentar. Mensaje claro al usuario y la conversación se cierra.

Experimento

Cambia la clave en .env, lanza **python main.py** y lee el campo error y la traza. Luego restaura la clave.

En el Sprint 13 llega HITL. Hoy basta con no tragarse el fallo en silencio.

La UI es un cliente, nunca el agente

□ Anti-patrón

```
if prompt := st.chat_input(...):  
    client.models.generate_content(...)
```

□ Lo que hace app.py

```
estado = procesar_turno(  
    st.session_state.agent_state, prompt)  
st.session_state.agent_state = estado
```

Si mañana cambias Streamlit por una API, **procesar_turno no se toca**. Solo cambia el cliente.

Cada interacción re-ejecuta todo el script



Nuestro while, dibujado como grafo

AgentState (dict)	State
preguntar_al_modelo · actualizar_estado · calcular_done	Nodes
if done or error: break	Edge condicional
run_demo completo	Graph
Estado inicial · return estado	START · END

LangGraph **no sustituye** diseñar buen estado y buenas paradas: las hace explícitas. En el Sprint 12 se programa.

CRITERIO DE CIERRE

Si puedes explicar esto en voz alta, has cerrado el sprint

- 1 Qué hace `procesar_turno(estado, mensaje)` y qué devuelve.
- 2 Quién decide `done`: el LLM o Python (`calcular_done`).
- 3 Por qué hace falta `max_turns` además de `done`.
- 4 Qué vive en `messages` y qué en `agent_state` en Streamlit.
- 5 Por qué este agente no inventa mediciones.

Ahora, el agente actúa.

Tools y function calling: el RAG pasa a ser una herramienta más, junto a una API y utilidades, con allowlist, max_steps y traza por tool.